FILE I/O & Process Management

Review of a system call,
File Descriptors,
Process Creation &
Termination,
Example Shell implementation

Review of a system call

System calls

User processes

- cannot perform privileged operations

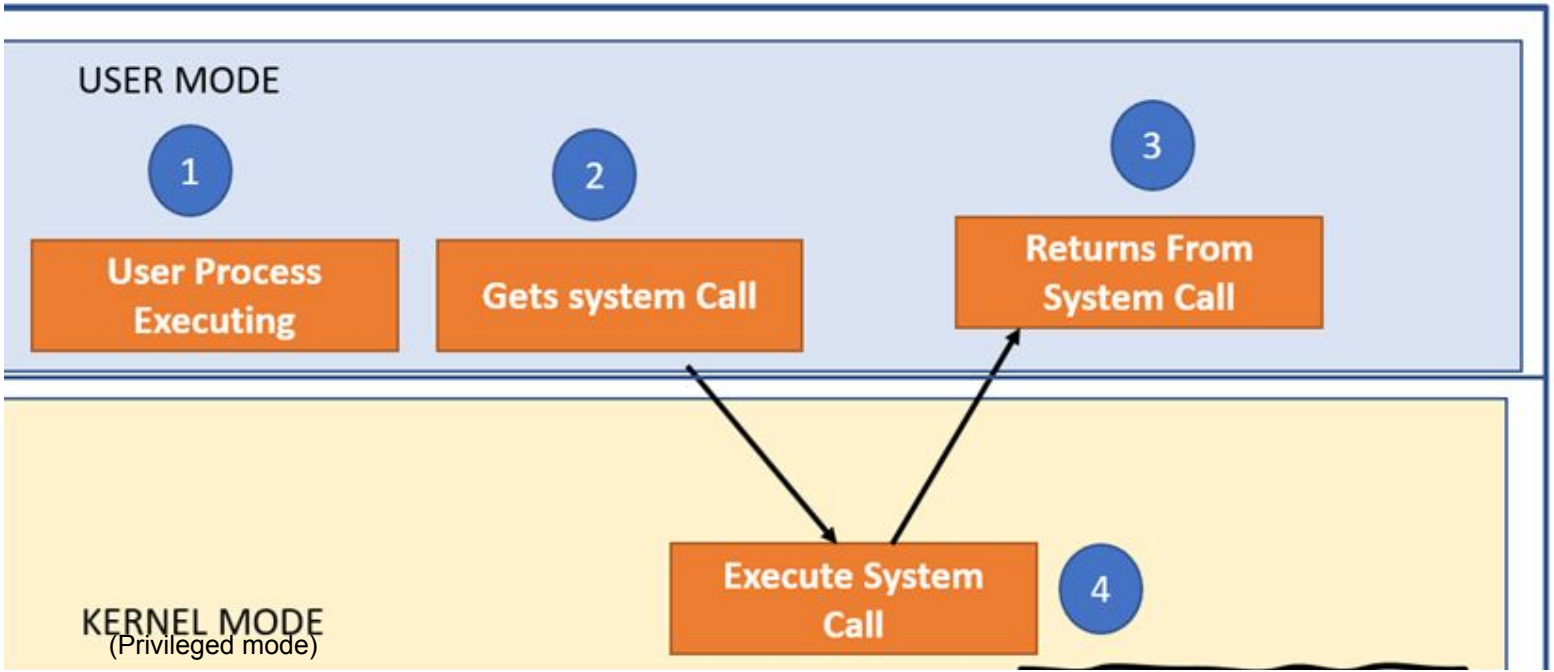
- request OS to do so on their behalf by issuing **system calls**

A **system call** is a request made by **an *active process (through traps or software interrupts)*** for a service performed by the kernel.

Examples:

- input/output (i.e., any movement of information to or from the combination of the CPU and main memory)
- creation of a new process.
- And many more https://man7.org/linux/man-pages/man2/syscalls.2.html

Why do we need a system call?



https://www.guru99.com/system-call-operating-system.html#2

How do we switch(transition) to kernel mode?

Arch/ABI	Instruction	System call #	Ret val	Ret val2	Error	Notes
alpha	callsys	v0	v0	a4	a3	1, 6
arc	trap0	r8	r0	-	-	
arm/OABI	swi NR	-	r0	-	-	2
arm/EABI	swi 0x0	r7	r0	r1	-	
arm64	svc #0	w8	x0	x1	-	
blackfin	excpt 0x0	P0	R0	-	-	
i386	**int $0x80**	**eax**	**eax**	**edx**	**-**	
ia64	break 0x100000	r15	r8	r9	r10	1, 6
m68k	trap #0	d0	d0	-	-	
microblaze	brki r14,8	r12	r3	-	-	
mips	syscall	v0	v0	v1	a3	1, 6
nios2	trap	r2	r2	-	r7	
…						
x86-64	**syscall**	**rax**	**rax**	**rdx**	**-**	**5**
x32	syscall	rax	rax	rdx	-	5
xtensa	syscall	a2	a2	-	-	

https://man7.org/linux/man-pages/man2/syscall.2.html

```c
#include <unistd.h>
int main(int argc, char *argv[]){
   write(1, "Hello World\n", 12); /* write "Hello World" to stdout */
   _exit(0);                      /* exit with error code 0 (no error) */
}
```

x86-32 equivalent assembly code

```
_start:
   movl $4, %eax   ; use the `write` [interrupt-flavor] system call
   movl $1, %ebx   ; write to stdout
   movl $msg, %ecx ; use string "Hello World"
   movl $12, %edx  ; write 12 characters
   int $0x80       ; make system call

   movl $1, %eax   ; use the `_exit` [interrupt-flavor] system call
   movl $0, %ebx   ; error code 0
   int $0x80       ; make system call
```

https://en.wikibooks.org/wiki/X86_Assembly/Interfacing_with_Linux

```c
#include <unistd.h>
int main(int argc, char *argv[]){
   write(1, "Hello World\n", 12); /* write "Hello World" to stdout */
   _exit(0);                      /* exit with error code 0 (no error) */
}
```

x86-64 equivalent assembly code

```
_start:
   movq $1, %rax    ; use the `write` [fast] syscall
   movq $1, %rdi    ; write to stdout
   movq $msg, %rsi  ; use string "Hello World"
   movq $12, %rdx   ; write 12 characters
   syscall          ; make syscall

   movq $60, %rax   ; use the `_exit` [fast] syscall
   movq $0, %rdi    ; error code 0
   syscall          ; make syscall
```

https://en.wikibooks.org/wiki/X86_Assembly/Interfacing_with_Linux

System-call without wrapper functions

In C, we use wrapper functions for system call APIs,

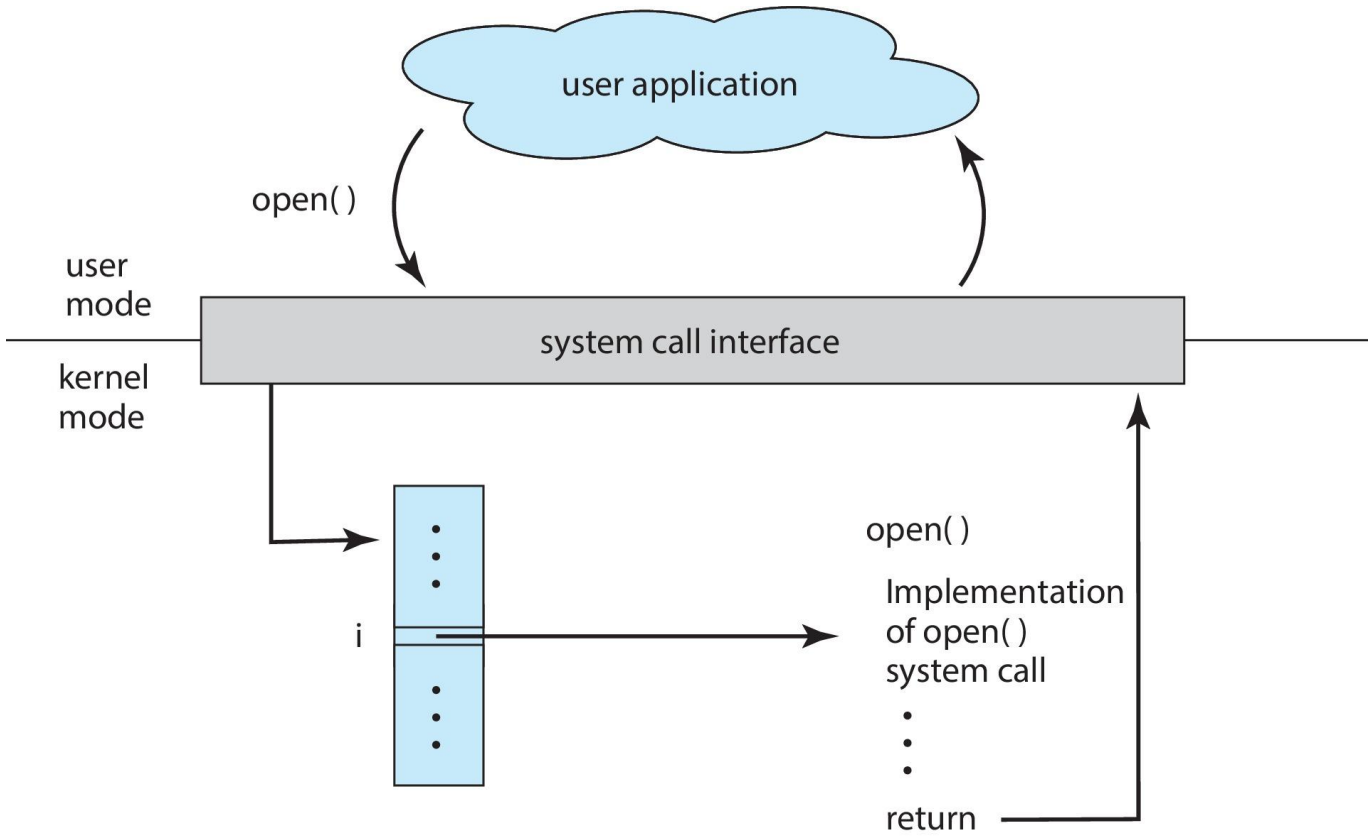
If there is no wrapper function, you can also call them with numbers.

```
 NAME   syscall - indirect system call

LIBRARY
      Standard C library (libc, -lc)
SYNOPSIS
      #include <sys/syscall.h>      /* Definition of SYS_* constants */
      #include <unistd.h>
      long syscall(long number, ...);
```

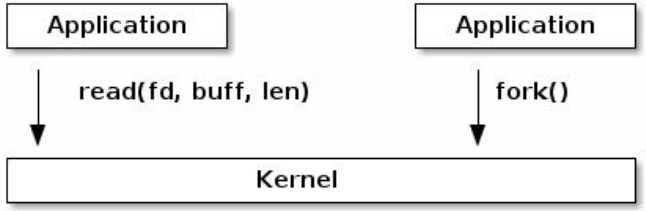
```c
/* from https://github.com/torvalds/linux/blob/master/include/linux/syscalls.h#L463 */
sys_write(unsigned int fd, const char __user *buf, size_t count);
```

```c
/* syscall() C lib function*/
#include <sys/syscall.h>      /* Definition of SYS_* constants */
#include <unistd.h>

int main(){
 char msg[] = "Hello, world!\n";
 syscall(SYS_write, 1, msg, 14);
 // syscall(1, 1, msg, 14); /*1 fd for stdout*/
 return 0;
}
```

API – System Call – OS Relationship

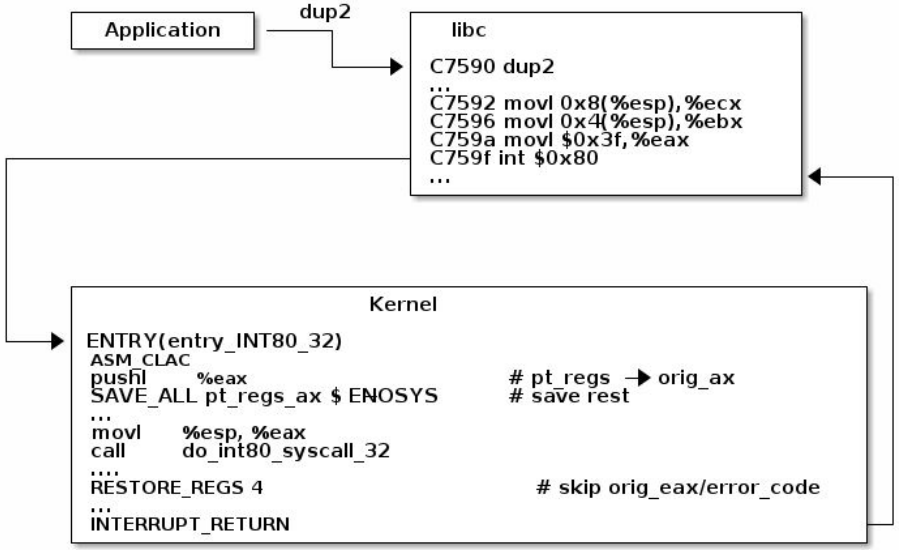


Example dup2() on x86



System calls are specific assembly instructions that

- setup information to identify the system call and its parameters
- trigger a kernel mode switch
- retrieve the result of the system call



https://linux-kernel-labs.github.io/refs/heads/master/lectures/syscalls.html

Some definitions

Interrupt

A signal to the kernel from a hardware or a software that indicates an event need “immediate” attention

- **Hardware interrupts:** the pressing of a key on the keyboard, a movement of the mouse.

- **Software interrupts (e.g. int 0x80)**
  - requests by an application program to operating systems
  - each interrupt is associated with an interrupt handler

★ CPU switch control to an interrupt handler
★ The execution of the program resumes after the interrupt is handled

trap vs exception vs interrupt

An exception is an unexpected event occurs because of **the execution of an instruction** (synchronous event).

- Division by zero
- **CPU detects this:**
- **Exception Types**
    - Traps,
        - Breakpoint, Trap, Syscalls, division by zero
    - faults,
        - Memory faults(Page fault),
    - aborts

Hardware interrupt is an unexpected event from outside the processor.

Software interrupt (e.g., int 0x80) is generated by the application programs(similar to traps).

https://stackoverflow.com/questions/3149175/what-is-the-difference-between-trap-and-interrupt

Summary and warnings

```
int file_fd = open(...);
```

- We got access to a file descriptor object.

```
write(file_fd, "Hello!", 6);
```

- writes some bytes to the file descriptor object that represents a file

A system call is an operation that the kernel carries out.

- First, the operating system prepares a system call.
- Next, the kernel executes the system call to the best of its ability in kernel space (a privileged operation).

★ System calls are expensive.
  - Their cost in terms of time and CPU cycles has recently been decreased, but try to use them as sparingly as possible.

★ this operation could fail for several reasons:
  - the file is no longer valid,
  - the hard drive failed,
  - the system was interrupted etc.

C system call-abstractions

Many C functions are abstractions that call the correct underlying system call

The implementations are different based on the platforms.

Common C functions

man pages are organized into sections:

Section 2 are system calls

```
man -S2 open
```

Section 3 are C libraries

```
man -S3 printf
```

File I/O: Universal I/O model

File descriptors

```
int file_fd = open(...);
```

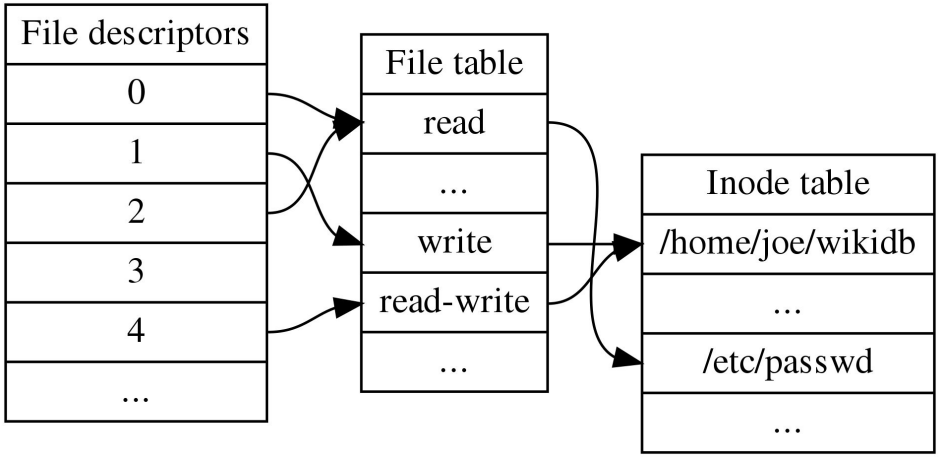
- opening a file returns a **file descriptor**,
  - An **integer** from **0** to **max**
  - Each process has
    - **0 STDIN_FILENO** for stdin
    - **1 STDOUT_FILENO** for stdout
    - **2 STDERR_FILENO** for stderr

subsequent operations (reading, writing, and so on) are done by using the file descriptor

```
write(file_fd, "Hello!", 6);
```

```
char c = 0;
read(file_fd, &c, 1);
```

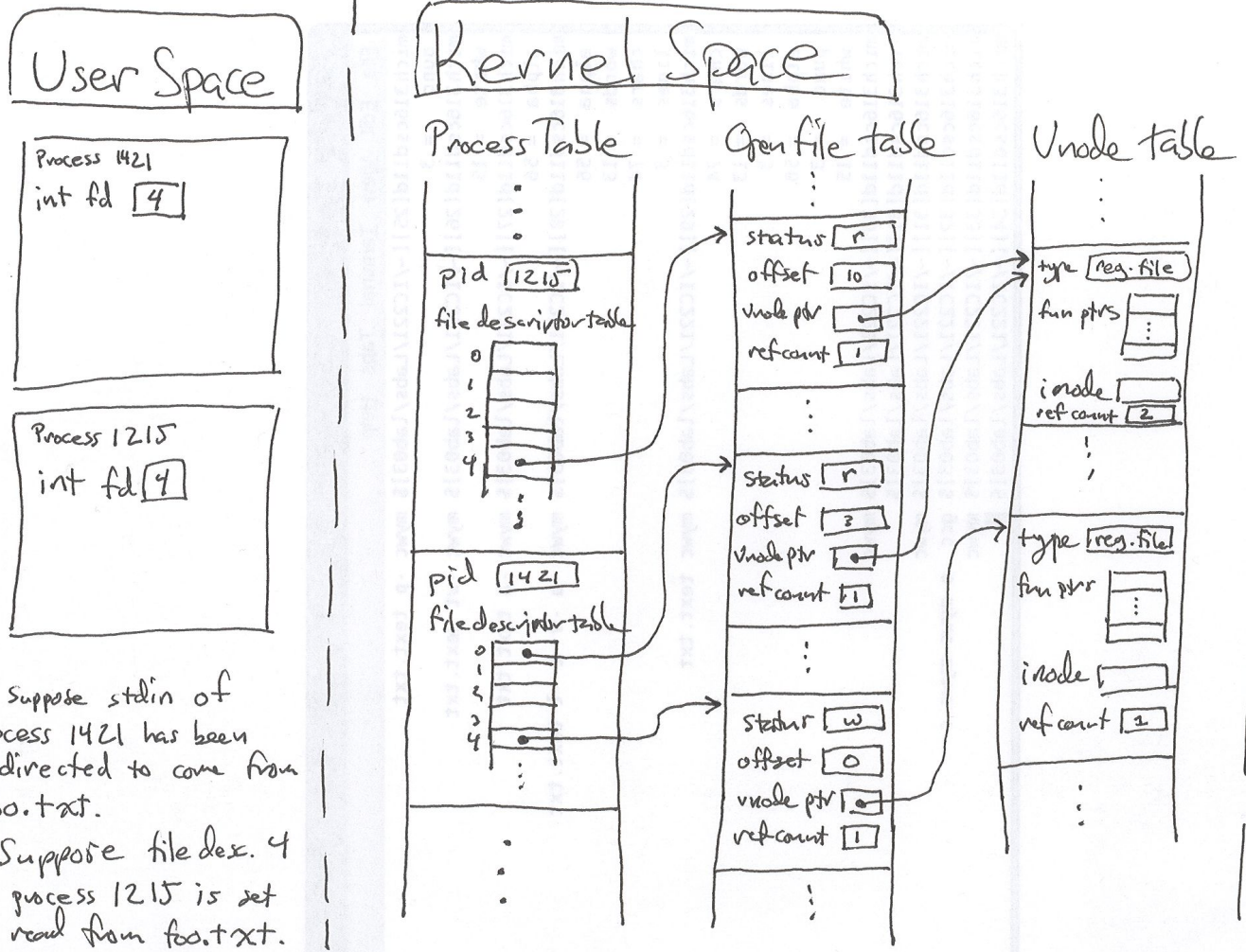
Fds for a single process



https://en.wikipedia.org/wiki/File_descriptor

- **file table:** table of the list of open files per-process
  - maintained by the kernel
  - **file descriptors(fds):** nonnegative integers that are used to index the entries of the file table.
  - Each entry in the list contains information about an open file,
    - including a pointer to an in-memory copy of the file's backing inode and associated metadata, such as the file position and access modes.

- Note that multiple file descriptors can refer to the same file table entry (dup system call),
- or multiple table can refer to the same inode table (same file opened multiple times).



- Suppose stdin of process 1421 has been redirected to come from foo.txt.
- Suppose file desc. 4 of process 1215 is set to read from foo.txt.

Taken from https://www.usna.edu/Users/cs/wcbrown/courses/IC221/classes/L09/Class.html

More on File descriptors

- File descriptors point to more than files
  - Remember “everything is file” in POSIX
  - Anything you can read or write (accessing devices)
- the operating system keeps track of them
- file descriptors may be reused between processes,
  - but inside of a process, they are unique.

- File descriptors may have a notion of position. These are known as **seekable streams**.
  - A program can read a file on disk completely because the OS keeps track of the position in the file, an attribute that belongs to your process as well.
- Other file descriptors point to network sockets and various other pieces of information, that are **non-seekable streams.**

Accessing files: open() system call

```
#include <fcntl.h>
int open(const char *path, int oflags);
int open(const char *path, int oflags, mode_t mode);
```

- O_RDONLY
  - Open the file so that it is read only.
- O_WRONLY
  - Open the file so that it is write only.
- O_RDWR
  - Open the file so that it can be read from and written to.
- O_APPEND
  - Append new information to the end of the file.
- O_TRUNC
  - Initially clear all data from the file.
- O_CREAT
  - If the file does not exist, create it. If the O_CREAT option is used, then you must include the third parameter.
- O_EXCL
  - Combined with the O_CREAT option, it ensures that the caller must create the file. If the file already exists, the call will fail.

Modes on O_CREAT

S_IRWXU 00700 user (file owner) has read, write, and execute permission

S_IRUSR 00400 user has read permission

S_IWUSR 00200 user has write permission

S_IXUSR 00100 user has execute permission

S_IRWXG 00070 group has read, write, and execute permission

S_IRGRP 00040 group has read permission

S_IWGRP 00020 group has write permission

S_IXGRP 00010 group has execute permission

S_IRWXO 00007 others have read, write, and execute permission

S_IROTH 00004 others have read permission

S_IWOTH 00002 others have write permission

S_IXOTH 00001 others have execute permission

The open() system call

```c
#include <fcntl.h>      /* open */
#include <sys/types.h>  /* needed on some UNIX */
#include <sys/stat.h>   /* needed on some UNIX */
#include <stdio.h>      /* perror */
int main(){

   int fd = open("file.txt", O_WRONLY | O_APPEND);
   if(fd == -1){
     perror("cannot open file: ");
     return -1;
   }
   return 0;
}
```

- O_RDONLY
  - Open the file so that it is read only.
- O_WRONLY
  - Open the file so that it is write only.
- O_RDWR
  - Open the file so that it can be read from and written to.
- O_APPEND
  - Append new information to the end of the file.
- O_TRUNC
  - Initially clear all data from the file.
- O_CREAT
  - If the file does not exist, create it. If the O_CREAT option is used, then you must include the third parameter.
- O_EXCL
  - Combined with the O_CREAT option, it ensures that the caller must create the file. If the file already exists, the call will fail.

fopen vs open

fopen() mode	open() flags
r	O_RDONLY
w	O_WRONLY | O_CREAT | O_TRUNC
a	O_WRONLY | O_CREAT | O_APPEND
r+	O_RDWR
w+	O_RDWR | O_CREAT | O_TRUNC
a+	O_RDWR | O_CREAT | O_APPEND

```
FILE *fdopen(int fd, const char *mode);
FILE *fopen(const char *path, const char *mode);
```

```
FILE *f = fopen("afile.txt", "r+");
```

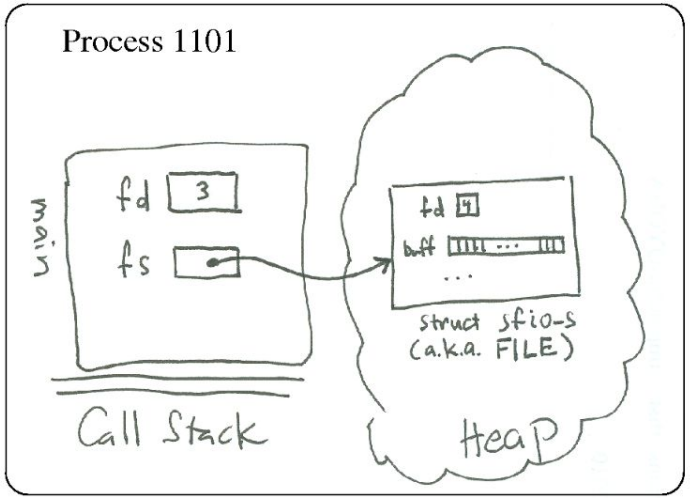
- ➔ Highlivel glibc function: stdio.h,
- ➔ Provides stream with buffer

```
int fd = open("afile.txt", O_RDWR);
```

- ➔ Low-level system call..
- ➔ You need to handle most stuff.

https://man7.org/linux/man-pages/man3/fopen.3.html

User Space

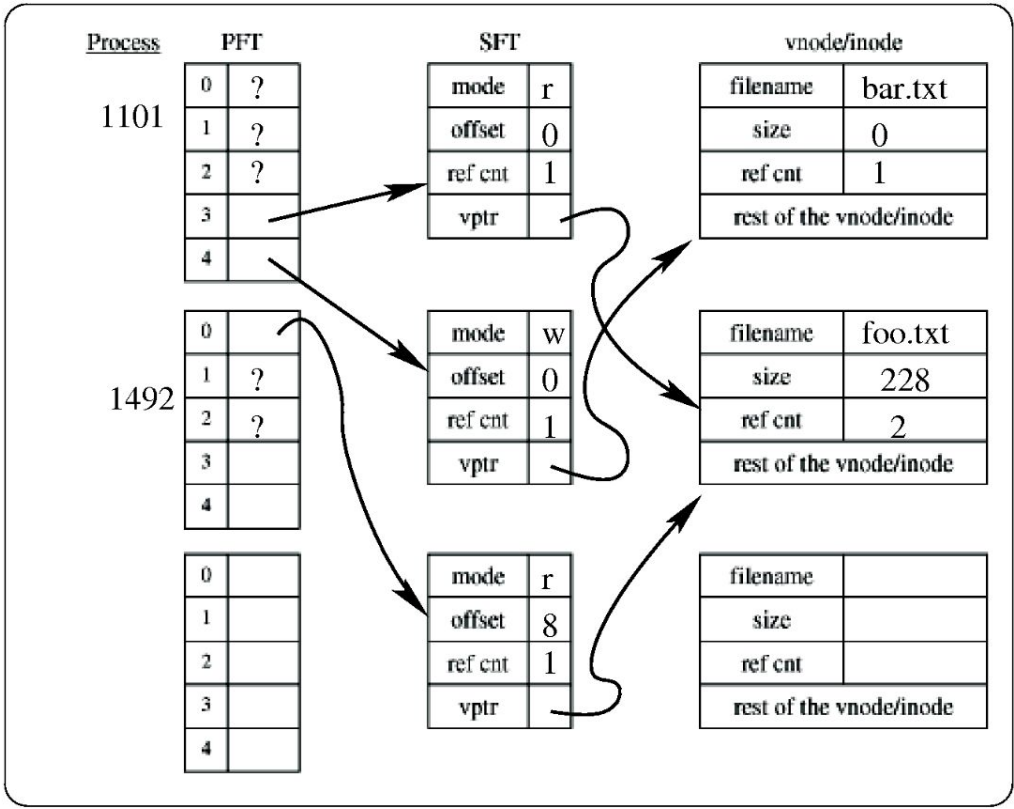



```
int main()
{
    int fd = open("foo.txt", O_RDONLY);
    FILE* fs = fopen("bar.txt", "w");
    ⋮
```

for I/O System Calls

for C standard library I/O

Kernel Space



Taken from https://www.usna.edu/Users/cs/wcbrown/courses/IC221/classes/L09/Class.html

read() system call

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);
```

- attempts to read up to **count** bytes from file descriptor **fd** into the memory-buffer pointed by **buf**.

➔ Returns a value indicating number of bytes read from the file

- `0` indicates `EOF`
- `-1` indicates `an error`
- `> 0`, indicates the number of bytes read

read() system call

```c
#include <fcntl.h>     /* open */
#include <sys/types.h>/* needed on some UNIX */
#include <sys/stat.h> /* needed on some UNIX */
#include <stdio.h>     /* perror */
#include <unistd.h>    /* read,close,write */
int main(){
   int fd = open("file.txt", O_RDONLY);
   if(fd == -1){
       perror("cannot open file: ");
       return -1;
   }
```

```c
   ssize_t ret; char buf[BUFSIZ];
   ret = read (fd, buf, BUFSIZ);
   if (ret == -1) {
       perror ("cannot read");
       close(fd);
       return -1;
   }

   buf[ret] = '\0';

   printf("%s", buf);
   close(fd);
   return 0;
}
```

write() system call

```
#include <unistd.h>    /* read, write, close */
ssize_t write (int fd, const void *buf, size_t len);
```

- writes up to **len bytes** from the memory pointed at by **buf**

➔ Returns a value indicating number of bytes written to file

- **>-1** indicates number of bytes written
- **-1** indicates an error

write() system call

```c
#include <fcntl.h>     /* open */
#include <sys/types.h> /* needed on some UNIX */
#include <sys/stat.h>  /* needed on some UNIX */
#include <stdio.h>     /* perror */
#include <unistd.h>    /* read, write, close */
#include <string.h>
int main(){
   int fd = open("file.txt",
                 O_WRONLY | O_APPEND | O_CREAT,
                 S_IRWXU);
   if (fd == -1){
       perror("cannot open file: ");
       return -1;
   }
   ssize_t ret;
   char buf[100] = "hello";
   int len = strlen(buf);
   ret = write(fd, buf, len);
   if (ret == -1){
       perror("cannot write!");
       close(fd);
       return -1;
   }
   else if (ret != len){
       printf(" written data "
              "less than len! error?");
   }
   close(fd);
   return 0;
}
```

Retrieving inode info: stat, fstat, system calls

```c
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>

int stat(const char *pathname, struct stat *statbuf);
int fstat(int fd, struct stat *statbuf);
int lstat(const char *pathname, struct stat *statbuf);

#include <fcntl.h>            /* Definition of AT_* constants */
#include <sys/stat.h>

int fstatat(int dirfd, const char *pathname, struct stat *statbuf, int flags);
```

```c
struct stat{
  dev_t st_dev;         /* ID of device containing file
*/
  ino_t st_ino;         /* Inode number */
  mode_t st_mode;       /* File type and mode */
  nlink_t st_nlink;     /* Number of hard links */
  uid_t st_uid;         /* User ID of owner */
  gid_t st_gid;         /* Group ID of owner */
  dev_t st_rdev;        /* Device ID (if special file) */
  off_t st_size;        /* Total size, in bytes */
  blksize_t st_blksize; /* Block size for filesystem I/O
*/
  blkcnt_t st_blocks;   /* Number of 512B blocks
allocated */

}
```

```c
#include <fcntl.h>     /* open */
#include <sys/stat.h>  /* stat, fstat */
#include <stdio.h>     /* perror */
#include <unistd.h>    /* read, write, close */
int main(){
   struct stat fd_stat;

   /* system call*/
   stat("file.txt", &fd_stat);
   printf("stat: size: %ld\n", fd_stat.st_size);

   /* or */
   int fd = open("file.txt", O_RDONLY);
   fstat(fd, &fd_stat);
   printf("fstat: size: %ld\n ", fd_stat.st_size);
   return 0;
}
```

repositioning the file offset

```
#include <sys/types.h>
#include <unistd.h>
off_t lseek(int fd, off_t offset, int whence);
```

whence:

- SEEK_SET
  - The file offset is set to offset bytes.
- SEEK_CUR
  - The file offset is set to **its current location + offset** bytes.
- SEEK_END
  - The file offset is set to **the size of the file + offset** bytes.

A motivating example (demo):
An intro to simple shell implementation

Resources:

https://ubuntu.com/tutorials/command-line-for-beginners#1-overview

https://man7.org/linux/man-pages/man3/system.3.html

https://www.tutorialspoint.com/c_standard_library/c_function_strtok.htm

https://man7.org/linux/man-pages/man3/strtok_r.3.html

- ➔ In the following, we will modify this with **fork/exec** and with **pipes.**
- ➔ In hw, you will complete the implementation.

An intro to simple shell implementation

```c
#include <stdlib.h>

int system(const char *command);
```

```c
#include <stdlib.h>
...
int status = system("./foo 1 2 3");
```

The system() library function

- uses fork(2) to create a child process that executes the shell command specified in command using execl(3)
- provides simplicity and convenience:
  - it handles all of the details of calling fork(2), execl(3), and waitpid(2), as well as the necessary manipulations of signals;
-

https://man7.org/linux/man-pages/man3/system.3.html

Process Management

Program files

A process is a running program.

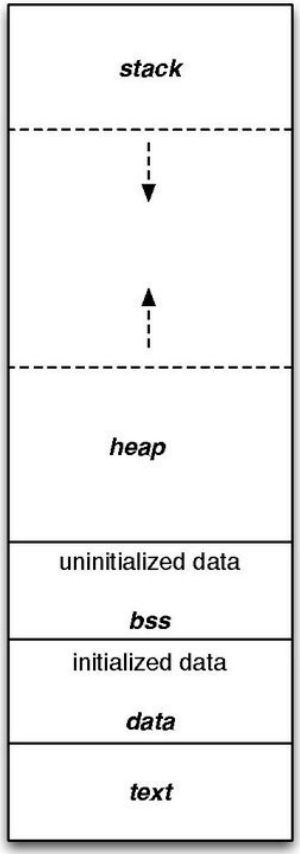
A program is a binary file that describes how to construct a process:

- Binary format identification: (metadata)
  - previously a.out (assembly out)
  - COFF (common object file format)
  - ELF (executable and linking format)
    - Currently used in most Unix
- Machine language instructions
  - Algorithm of the program
- Program entry-point address
  - Execution point
- Data: literal constants
- Libraries to link and where to fill in the address of those libraries
- Other info that describes how to construct process

Process Content

Processes are isolated! Each process gets:

- A stack
  - The stack is the place where automatically allocated variables and function call return addresses are stored.
- A heap
  - Manually controlled memory allocation for objects (malloc)
- A data segment
  - Initialized data segment
    - int global = 1
  - Uninitialized data segment / BSS
    - int global; //assumed to be zero
- Text segment
  - Executable instructions
  - Readable but not writable



Other contents:

- Running state
- File descriptors
- Permissions
- Arguments
- Environment variables

```c
/*taken from the book linux programming interface*/
#include <stdio.h> #include <stdlib.h>
char globbuf[65536];              /* Uninitialized data segment */
int primes[] = {2, 3, 5, 7};      /* Initialized data segment */

static int square(int x) {        /* Allocated in frame for square() */
   int result;                    /* Allocated in frame for square() */
   result = x * x;
   return result;                 /* Return value passed via register */
}

static void docalc(int val) {     /* Allocated in frame for docalc() */
   printf("The square of %d is %d\n", val, square(val));
   if (val < 1000) {

       int t;                     /* Allocated in frame for doCalc() */
       t = val * val * val;
       printf("The cube of %d is %d\n", val, t);
   }
}
```

```c
int main(int argc, char *argv[]){ /* Allocated in frame for main() */
   static int key = 9973;       /* Initialized data segment */
   static char mbuf[10240000]; /* Uninitialized data segment */
   char *p;                     /* Allocated in frame for main() */
   p = malloc(1024);            /* Points to memory in heap segment */

   docalc(key);
   exit(EXIT_SUCCESS);
}
```

Process ID (pid)

Each process is represented by a unique identifier

To list processes in Linux

```
$ps aux

$ top
```

- The first process kernel executes after booting is init process with pid 1
- When there are no other processes running, kernel runs idle process with pid 0
- For other processes, pid is allocated in a linear fashion

The getpid() system call

Returns the process ID of the calling process

- The process id is represented by **pid_t**
  - **A typedef to C int type**

```
#include <unistd.h>     /* pid_t getpid (void); */
#include <sys/types.h>  /* maybe needed in some UNIX*/
#include <stdio.h>
int main(){

   pid_t pid = getpid();
   printf("process id: %d\n", pid);
   return 0;
}
```

Process Hierarchy

- A process that spawn a new process is called parent process
  - The new process is called child process
  - Child process inherits the parent's user and group ownerships
  -
- Each process has a parent process id (ppid)
  - getppid() returns parent process id of the calling process

```c
#include <unistd.h>     /* pid_t getppid (void); */
#include <sys/types.h>  /* needed in some UNIX*/
#include <stdio.h>
int main(){

   pid_t pid = getpid();
   pid_t ppid = getppid();
   printf("process id: %d\n", pid);
   printf("process id: %d\n", ppid);
   return 0;
}
```

Running a new process: exec family system calls

- The exec family of functions replaces the current running process with a new process.
- They are declared in **<unistd.h>**

int execl (const char *path,const char *arg, ...);

- Arg is the first argument to the program
- The list of arguments must be **(char *) NULL** terminated

```
#include <unistd.h>
#include <stdio.h>      /* perror */
int main(){

  int ret;
  ret = execl("/usr/bin/vim", "vim",  NULL);
  if (ret == -1)
      perror("execl");
}
```

The rest of exec family

```
#include <unistd.h>
int execlp (const char *file, const char *arg, ...);

int execle (const char *path, const char *arg,...,char * const envp[]);

int execv (const char *path, char *const argv[]);

int execvp (const char *file, char *const argv[]);

int execve (const char *filename,char *const argv[], char *const envp[]);
```

Original system call, the others are wrapper functions

- v arguments are given as vector (array)
- l arguments are given as a list
- e denotes a new environment

```c
/*program1*/
#include<stdio.h>
#include<unistd.h>
int main(){
   printf("program1 called by execv()\n");
   return 0;
}
```

```c
/*program2*/
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
int main() {
   printf("program2\n");
   char *args[]={"./program1",NULL};
   int ret = execv(args[0],args);

   /* the rest is not executed!!!*/

   if (ret == -1) perror("cannot execute: ");
   printf("end of program2-----");
   return 0;
}
```

execve() executes the program referred to by pathname **(see $man execve)**.

- This causes the program that is currently being run by the calling process to be replaced with a new program, with newly initialized stack, heap, and (initialized and uninitialized) data segments.
- execve() does not return on success, and the text, initialized data, uninitialized data (bss), and stack of the calling process are overwritten according to the contents of the newly loaded program.

Modifying shell implementation with execve() (demo)

★ With only exec, we have the limitation of a single process

➔ Next: Use a new child process

Creating a new process: fork() system call

```
pid_t fork (void);
```

- Creates a new process by **cloning** the current process
- Declared in `<unistd.h>`

```
#include <sys/types.h>
#include <unistd.h>
int main{
   pid_t pid;
   pid = fork ();
   if (pid > 0)
       printf ("I am the parent of pid %d!\n", pid);
   else if (pid == 0)
       printf ("I am the child!\n");
   else if (pid == -1)
      perror ("fork");
}
```

```c
#include  <stdio.h>
#include  <sys/types.h>
#include <unistd.h>

int  main(){
   pid_t  pid = getpid();
   printf("parent process id: %d\n", pid);

   fork();

   /*this part executed by both child and parent*/
   pid = getpid();
   printf("hello from pid %d\n", pid);
   return 0;

}
```

- two identical copies of address spaces,
  - one for the parent and the other for the child.
- The child inherits copies of the parent's set of open file descriptors.
- Both starts their execution at the next statement following the fork() call.

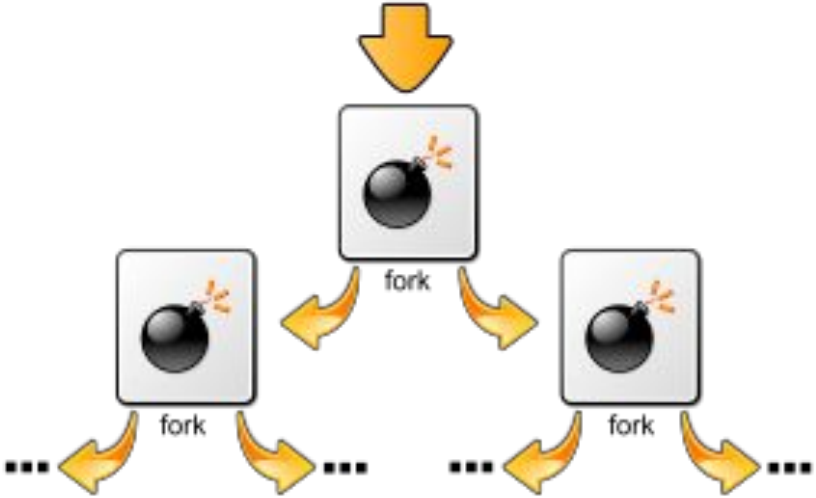
How many child processes?

```
#include  <stdio.h>
#include  <sys/types.h>
#include <unistd.h>
int  main(){
   pid_t  pid = getpid();
   printf("parent process id: %d\n", pid);

   fork();
   fork();

   pid = getpid();
   printf("hello from pid %d\n", pid);
   return 0;
}
```

How many child processes?

```
#include  <stdio.h>
#include  <sys/types.h>
#include <unistd.h>
int  main(){
   pid_t  pid = getpid();
   printf("parent process id: %d\n", pid);

   fork();
   fork();
   fork();
   fork();
   pid = getpid();
   printf("hello from pid %d\n", pid);
   return 0;
}
```

fork-bomb!



https://en.wikipedia.org/wiki/Fork_bomb

Modifying shell implementation with fork() (demo)

- ★ Parent process needs to know the status of a child
  - Was it successful
    - exit(0)
  - Failed
    - (exit(-1))

➔ Next: “waiting” for a child

Waiting for terminated child process

```
#include <sys/wait.h>

pid_t wait(int *status);
```

- To wait child process to finish or change its status
  - Child terminated
  - Child was stopped by a signal
  - Child was resumed by a signal
- It **suspends execution of the calling process**

Waiting for terminated child process

```
pid_t wait(int *status);
```

➔ Use **WIF..()** to determine the cause.

```
int WIFEXITED (status);
```

- Returns true if process terminated normally

```
int WIFSIGNALED (status);
```

- Returns true if signal causes process termination

```
int WIFCONTINUED(status);
```

- Returns true if process was continued

```
int WIFSTOPPED (status);
```

- Returns true if process was stopped

➔ Use **W..** to get the status value

```
int WEXITSTATUS (status);
```

- Returns 8-bits that were passed to _exit()

```
int WTERMSIG (status);
```

- Number of signal that caused the termination

```
int WSTOPSIG (status);
```

- If stopped by signal returns number of signal

```
int WCOREDUMP (status);
```

- Returns true if process dumped core

```c
#include <unistd.h>#include <stdio.h>#include <sys/types.h>#include <sys/wait.h>
int main(){
   int status;
   pid_t pid;
   if (!fork()) return 1;
   pid = wait(&status);
   if (pid == -1) perror("wait returns -1:");
   if (WIFEXITED(status))
       printf("Normal termination with exit status=%d\n",
              WEXITSTATUS(status));
   if (WIFSIGNALED(status))
       printf("Killed by signal=%d%s\n",
              WTERMSIG(status), WCOREDUMP(status) ? "(dumped core)" : "");
   if (WIFSTOPPED(status))
       printf("Stopped by signal=%d\n",
              WSTOPSIG(status));
   if (WIFCONTINUED(status)) printf("Continued\n");
   return 0;
}
```

Waiting for specific process

```
pid_t waitpid(pid_t pid, int *status, int options);
int waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options);
```

pid

- **-1**: wait for any child process

  waitpid(-1,&status,0);
  - equivalent to wait
- **> 0**: wait for the child PID = **pid**.
- **0**: wait for any child with group ID = that of the calling process.
- **< -1**: wait for any child with group ID = **|pid|**.

options

- WNOHANG – Return the status information immediately. If status is not changed, returns 0
- WCONTINUED – Wait for continued children

With waitid function

- *WNOWAIT – Wait, but leave the child wait-able by another wait call*
- *WEXITED – Wait for terminated children*
- *WSTOPPED – Wait for stopped children*

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
int main() {
   pid_t cpid, w;
   int status;
   if ((cpid = fork()) < 0){
       perror("fork() error");
       exit(EXIT_FAILURE);
   }
   if(cpid == 0) {/*child process*/
       sleep(5);
       exit(1);
   }
```

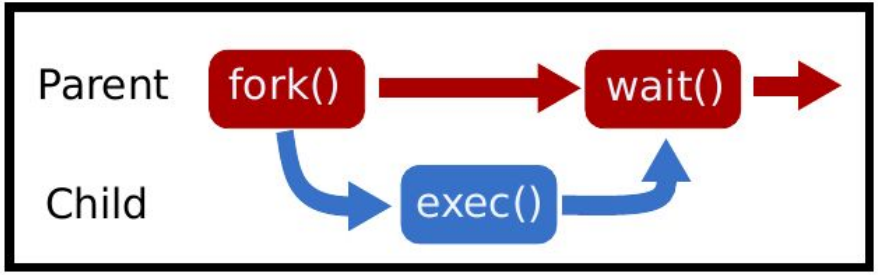
```c
    else do { /*parent process*/
      w = waitpid(cpid, &status, WNOHANG);
       if(w == -1){ perror("waitpid"); exit(EXIT_FAILURE);}

       else if(w == 0){
           /*child termination info is not available*/
           printf("child is still running\n");
           sleep(1);
       } else{
           if (WIFEXITED(status)){
               printf("child exit status %d\n",
                 WEXITSTATUS(status));
           } else{
               printf("child did not exit successfully\n");
           }
       }
   } while (w == 0);
   exit(EXIT_SUCCESS);
}
```

Sorting while sleeping:)
Exercise: run this after the class and try to understand

```c
/*sleep sort*/
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main(int argc, char *argv[]){
   while (--argc > 1 && !fork());
   int val = atoi(argv[argc]);
   sleep(val);
   printf("%d\n", val);
   wait(0);
   return 0;
}
```

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
int main() {
    pid_t pid = fork();
    if (pid < 0) {/* fork failure*/
        exit(1);
    } else if (pid > 0) {
        waitpid(pid, NULL, 0);
    } else {
        execl("/bin/ls", "/bin/ls", NULL);
        exit(1); /*For safety.*/
    }
}
```

fork-exec-wait



```c
#define HELLO_NUMBER 10
int main(){
    pid_t children[HELLO_NUMBER];
    int i;
    for (i = 0; i < HELLO_NUMBER; i++){
        pid_t child = fork();
        if (child == -1) break;
        if (child == 0){
            /*fails! misspelled echo*/
            execlp("ehco", "echo", "hello", NULL);
        }else{
            /* Parent */
            children[i] = child;
        }
    }
    int j;
    for (j = 0; j < i; j++){
        waitpid(children[j], NULL, 0);
    }
    return 0;
}
```

Fork warning!

- How many processes are created when execlp fails?

Fork and file descriptors

- ★ **File descriptors are copied in fork!**
  - **Each processes fds point to same entry in open file table**
  - file pointers in file objects are shared between the child and the parent, **lseek()** on a descriptor in the child process can affect a subsequent read or write by the parent.

```
int file = open(...);
if(!fork()) {
    read(file, ...);
} else {
    read(file, ...);
}
```

★ one process reads the first part, the other process reads the following part

Fork and file descriptors

```
int file = open(...);
if(!fork()) {
   read(file, ...);
} else {
   read(file, ...);
}
```

★ one process reads one part the other reads another part

```
if(!fork()) {
   int file = open(...);
   read(file, ...);
} else {
   int file = open(...);
   read(file, ...);
}
```

★ we have two file descriptors, file pointers in file objects are not shared.

Example

A file named mytemp

```
1....line
2....line
3....line
4....line
```

```c
int fd = open("mytemp", O_RDONLY);
pid_t pid = fork();
char buf[20] = {0};
if (pid == 0) {
    read(fd, buf, 10);
    printf("child:%.10s\n", buf);
} else if (pid > 0) {
    wait(NULL);
    read(fd, buf, 10);
    printf("parent:%.10s\n", buf);
}
```

```
child:1....line
parent:2....line
```

```c
pid_t pid = fork();
int fd = open("mytemp", O_RDONLY);
char buf[20] = {0};
if (pid == 0) {
    read(fd, buf, 10);
    printf("child:%.10s\n", buf);
} else if (pid > 0) {
    wait(NULL);
    read(fd, buf, 10);
    printf("parent:%.10s\n", buf);
}
```

```
child:1....line
parent:1....line
```

Redirection with file descriptors and limitations

Modifying shell implementation (demo)

★ In HW, you will finalize this with pipes

Zombie processes

- It is good practice to wait on child-processes.
- **Zombie process**
  - Although child-process terminates, it still takes up a spot in the kernel process table for your process:
  - The process table keeps track of the following information about this process:
    - **PID, status, and how it was killed.**
- The only way to get rid of a zombie is to wait on this child process.
  - If a long-running parent never waits for your children, it may lose the ability to fork.

Orphan processes

- Parent process can continue to execute code without having to wait for the child process.
- If a parent dies without waiting on its children, a process can orphan its children.
  - **init process becomes the new parent process**
    - init is the first process, whose **PID is 1.**
    - **Try getppid()**
- These orphans will eventually finish and for a brief moment become a zombie.
- The init process automatically waits for all of its children, thus removing these zombies from the system.

Why multi-process programming?

Another program can be used to accomplish the desired task

- Exec family functions

Parent program needs to continue running while another process services/waits for some requests

- Network based server programs

Multi-thread vs multi-process

- process has its own address space
- more task isolation and independence
- we‘ll discuss more after pthreads